		Baseline	**i2p**(**p2i** p) fold disabled	Sound	add **psub**
`-O0`	Total	21,219,982	21,219,982	21,219,982	21,194,610
	inttoptr	1,554	1,554	1,554	1,554
	ptrtoint	39,726	39,726	39,726	14,338
	psub	0	0	0	12,806
`-O3`	Total	15,015,064	15,030,020	14,950,405	14,750,101
	inttoptr	29,976	44,255	2,580	2,573
	ptrtoint	38,249	45,425	64,585	9,431
	psub	0	0	0	29,117

Table 1. Number of instructions with different prototypes

In the last column, we show the data for a prototype that uses the new **psub** instruction. Not only are the number of casts reduced in the non-optimized build (since we changed Clang to lower pointer subtraction into **psub**), but the optimized build also shows a reduction since we also added support for the optimizers to create **psub** when needed.

Finally, we note the number of cast instructions is still larger in the optimized build since optimizations like loop unrolling and inlining can soundly duplicate instructions.

6.4 Compile time

We measured the time it takes to compile each file of all the benchmarks in an optimized build. We exclude files with a very small compile time (< 0.15 s) in these results.

In the first prototype, the average compile time had a 0.1% slowdown. The worst slowdown was 10.5% and the best speedup 13.9%. For the second prototype, we observed an average slowdown of compilation time of 1.1%. The worst slowdown increased to 9.3%. A reason for the second prototype regressing further on the compilation time is due to extra analysis required to propagate pointer equalities. Also, our prototype implementations were not optimized to reduce compilation time.

7 RELATED WORK

Much work has been done on defining and clarifying the semantics of C, including the semantics of pointer provenance [Chisnall et al. 2016; Hathhorn et al. 2015; Krebbers 2013; Krebbers and Wiedijk 2015; Memarian et al. 2016; Memarian and Sewell 2016a,b]. However, focusing on source language semantics involves different considerations than that of a compiler IR. For example, many of the optimizations presented in this paper are not sound under the proposed C semantics. That is not a problem, as long as C can efficiently be compiled to an IR like LLVM's that allows the desired optimizations.

CompCert [Leroy 2009; Leroy and Blazy 2008] is a compiler for a subset of C that is formalized in Coq. It defines a pointer as a pair of block id and offset, which is equivalent to our logical pointer. CompCert does not, however, support pointer-to-integer casts, thereby avoiding the problems we address in this paper. Ševčík et al. [2013] extend CompCert with a weak memory model.

Vellvm [Zhao et al. 2012] formalizes the LLVM IR in Coq. It adopts CompCert's memory model, inheriting its weaknesses.

Besson et al. [2014; 2015; 2017a; 2017b] propose an extension of the CompCert memory model supporting casts between integers and pointers. When a pointer is cast to an integer, a fresh symbol is created to represent the underlying physical address and henceforth keep the result of arithmetic or bitwise operations over it as symbolic expressions. When a concrete value is needed for a symbolic expression, if it uniquely determines an integer value, then that value is returned; otherwise, poison is returned. They verify CompCert optimizations and programs performing bitwise operations on pointers with their memory model. However, their model is limited in practice